

Programación Concurrente

Trabajo Práctico

Base de datos no SQL Replicada (en memoria)

Entrega: 21 de Febrero de 2025

Se desea desarrollar en Erlang un almacenamiento replicado (en memoria) inspirado en las bases de datos distribuidas no relacionales como Dynamo [1] y Cassandra [2]. Estas bases de datos consisten en distintas réplicas que almacenan pares clave-valor que pueden accederse mediante dos operaciones: **get**, para recuperar el valor asociado a una clave, **put** para almacenar el valor de una clave en particular y **rem** para eliminar una clave. Un cliente, realiza una operación sobre una de las réplicas, que actúa como coordinador y media entre el cliente y las restantes réplicas. Cada solicitud del cliente está asociada con un nivel de consistencia que especifica el grado de consistencia requerido sobre esas operaciones. Para una operación **put** o **rem**, el nivel de consistencia indica el número de réplicas que deben escribirse antes de enviar la confirmación al cliente. De manera similar, el nivel de consistencia de una operación **get** especifica el número de réplicas que deben responder a la solicitud de lectura antes de devolver los datos al cliente.

Ejercicio preparatorio: Definir en Erlang un módulo `base_no_replicada` que implementa un proceso que mantiene un diccionario, es decir, una colección de pares clave/valor. Las claves deben ser únicas en el diccionario, aunque no es necesario que los valores lo sean. Cada valor estará acompañado por un timestamp. El módulo debería exportar las siguientes operaciones:

- **start/1:** Inicia el proceso del servidor de diccionario y lo registra con el nombre pasado por argumento. Si ya hay un servidor con el mismo nombre, debería fallar.
- **stop/0:** Detiene el proceso del servidor si está en ejecución. Si no hay un proceso de servidor de diccionario en ejecución, esta función no tiene efecto.
- **put/3:** Toma una clave, un valor y un timestamp, y agrega la clave y su valor al diccionario con el timestamp asociado. Si la clave dada ya está en el diccionario, el nuevo valor reemplaza al antiguo sólo si el timestamp es mayor al del valor en el diccionario. En caso contrario, no lo cambia. Esta función devuelve `ok` o `ko` dependiendo de el cambio tuvo efecto o no.
- **rem/2:** Toma una clave y un timestamp, y elimina lógicamente al elemento del diccionario, sólo si el timestamp es mayor al del valor en el diccionario. Si el elemento no está presente, la operación debe recordar que el elemento fue eliminado con el timestamp dado, de manera tal que inserciones con timestamp menor no tengan efecto en el futuro. Esta función devuelve `ok` si la clave se eliminó correctamente, o `notfound` si la clave no estaba en el diccionario (ya sea lógica o físicamente).
- **get/1:** Toma una clave y encuentra el valor correspondiente en el diccionario. Si la clave está en el diccionario, esta función devuelve la tupla `ok, Valor, TimeStamp`, donde Valor es el valor asociado con la clave y el TimeStamp asociado. Si el valor ha sido eliminado, devuelve `ko, TimeStamp`. Si la clave no está en el diccionario, esta función devuelve `notfound`.
- **size/0:** Devuelve un entero que especifica el número de claves en el diccionario.

Utilizar envío de mensajes para la implementación de las operaciones, aún cuando los clientes no necesiten ni recibir mensajes explícitamente y, por lo tanto, no necesitarán conocer el formato de esos mensajes.

Versión Replicada Una base replicada va a estar conformada por `n` procesos que se comportan idénticamente. El valor `n` está definido al momento de creación de la base de datos. Se deberán implementar los siguientes módulos:

- Módulo `base_de_datos` que provee operaciones para la creación de una base de datos replicadas.
 - **start/2:** Recibe `nombre` y la cantidad de réplicas `n`. Deberá iniciar `n` réplicas, cada una de las cuales se registrará con el nombre `nombre-i`, donde `i` es un natural menor o igual a `n`. Todas las réplicas iniciarán en el estado vacío y conocerán al conjunto de réplicas restantes.
 - **stop/0:** Envía el mensaje `stop` a todas las réplicas
- Módulo `replica`: Deberá proveer las siguientes operaciones.

- **start/2**: Recibe **nombre** y una lista con los nombres de las restantes réplicas. Deberá registrar un servicio con su nombre e inicializar el diccionario como vacío.
- **stop/1**: Toma el nombre de la réplica y detiene al servicio asociado. Si no hay un proceso de servidor de diccionario en ejecución, esta función no tiene efecto.

Las operaciones de manipulación de la base de datos son análogas al ejercicio anterior, sólo que reciben dos parámetros adicionales:

- la **identidad de la réplica** que coordinará la operación, y
- el **nivel de consistencia**. En este caso consideraremos:
 - ◊ **one**: significa que la operación sólo necesita confirmación de la réplica que coordina la operación (no se deben esperar respuestas de otras réplicas).
 - ◊ **quorum**: la operación requiere confirmación de al menos la mitad de las réplicas.
 - ◊ **all**: la operación requiere confirmación de todas las restantes réplicas.

El funcionamiento para cada operación puede ser descripto de la siguiente manera: cuando una réplica recibe un pedido de operación (**put**, **rem**, **get**), el pedido se reenvía a las restantes réplicas (utilizando comunicación por mensajes). Dependiendo del nivel de consistencia, la réplica espera las respuestas necesarias antes de confirmar la operación al cliente. Es decir, si el nivel de consistencia es

- **one**, se realiza la operación localmente y se confirma el resultado al cliente sin esperar respuesta de las restantes réplicas.
 - **quorum**, se esperan respuestas de al menos la mitad de las réplicas antes de confirmar el resultado al cliente. En este caso, el resultado final que se devuelve al usuario corresponde a la respuesta que tiene el mayor timestamp.
 - **all**, es similar a la anterior, pero requiere la respuesta de todas las réplicas.
- **put/5**: Toma una clave, un valor, un timestamp, un nivel de consistencia y la identidad de la réplica coordinadora. Esta operación inicia un insert en la réplica coordinadora, que luego de recibir el pedido, la reenviará a las restantes réplicas. Dependiendo del nivel de consistencia, esperará las respuestas necesarias de las restantes réplicas antes responder a la solicitud del usuario.
 - **rem/4**: Toma una clave, un timestamp, el nivel de consistencia y la identidad de la replica que coordinará. Como en el caso anterior, reenviará la solicitud y esperará las respuestas necesarias.
 - **get/3**: Toma una clave, el nivel de consistencia y la identidad de la réplica. El funcionamiento es análogo a los anteriores.

Correctitud: Pensar si su solución permite que las réplicas manejen pedidos de manera concurrente. Pensar qué sucede si una réplica está haciendo un **insert** con consistencia **all**. ¿Puede tratar otras solicitudes mientras espera respuestas de otras réplicas? ¿Es su solución libre de deadlocks? Pensar en operaciones que se inician en distintas réplicas.

Testing: Se espera que propongan tests de unidad para los distintos módulos desarrollados.

1. Condiciones de Entrega

El trabajo se realizará en grupos de 2 (dos) ó 3 (tres) integrantes. Se aceptarán grupos de 4 (cuatro) integrantes pero la defensa será de a dos integrantes.

La entrega deberá incluir:

- Código fuente.
- Reporte que describa la estrategia principal de la solución.

2. Bibliografía

1. Sivasubramanian, S. (2012, May). Amazon dynamoDB: a seamlessly scalable non-relational database service. In Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data (pp. 729-730).
2. Lakshman, A., & Malik, P. (2010). Cassandra: a decentralized structured storage system. ACM SIGOPS operating systems review, 44(2), 35-40.